



DG INTERNS HUB

VAPT PROJECT REPORT

Web Security & Vulnerability Assessment / Penetration Testing

Target Application: DVWA (Damn Vulnerable Web Application)

Prepared as part of Cybersecurity Internship

Week 6: Web Security & Vulnerability Testing (VAPT Basics)

DG Interns Hub

Submitted by: Avisha Masih

Cybersecurity Intern | SIN: DG/AUGUST/CYBER/098

Submission Date: 30 September 2026

Table of Contents

1. Introduction to VAPT
2. Objective
3. Tools Used
4. Testing Environment Setup
5. Testing Process
 - 5.1 SQL Injection Testing
 - 5.2 Cross-Site Scripting (XSS) — Reflected
 - 5.3 Cross-Site Scripting (XSS) — Stored
 - 5.4 Automated Scanning with OWASP ZAP
6. Findings & Risk Levels
7. Mitigation Methods
8. Conclusion

1. Introduction to VAPT

Vulnerability Assessment and Penetration Testing (VAPT) is a structured security-testing process used to identify, analyse, and address weaknesses in a web application before they can be exploited by malicious actors. A Vulnerability Assessment focuses on discovering and cataloguing potential weaknesses, typically using automated scanning tools, while Penetration Testing goes a step further by actively attempting to exploit those weaknesses in a controlled manner to demonstrate real-world impact.

This report documents a VAPT exercise carried out on DVWA (Damn Vulnerable Web Application) — a PHP/MySQL web application that is deliberately built with common security flaws so that testers can practice identifying and exploiting them in a safe, legal, and isolated environment. The exercise combines manual testing using Burp Suite with automated scanning using OWASP ZAP, reflecting the two complementary approaches used in real-world security assessments.

2. Objective

- To understand and practically demonstrate common web application vulnerabilities listed in the OWASP Top 10.
- To set up an isolated, legal testing environment using DVWA on Kali Linux.
- To perform manual vulnerability testing (SQL Injection, Cross-Site Scripting) using Burp Suite for request interception and analysis.
- To perform automated vulnerability scanning using OWASP ZAP and interpret the results.
- To document findings with evidence (screenshots), assign risk levels, and recommend mitigation measures.

3. Tools Used

DVWA (Damn Vulnerable Web Application)

An intentionally vulnerable PHP/MySQL web application used as the target for this exercise. It provides multiple modules (SQL Injection, XSS, Brute Force, CSRF, File Upload, etc.) with adjustable security levels (Low, Medium, High, Impossible), making it ideal for learning how vulnerabilities work and how they are fixed. For this exercise, the Security Level was set to Low to clearly demonstrate each vulnerability.

Burp Suite (Community Edition)

An intercepting proxy used for manual web application security testing. It sits between the browser and the target application, allowing every HTTP request and response to be captured, inspected, and modified before being forwarded. It was used in this exercise to intercept requests, observe how the SQL Injection payloads were transmitted, and examine full request/response pairs via the HTTP History feature.

OWASP ZAP (Zed Attack Proxy)

A free, open-source Dynamic Application Security Testing (DAST) tool maintained under the OWASP Foundation (now developed by Checkmarx). It was used to run an Automated Scan against DVWA, which combines spidering (crawling the site to discover pages) with active and passive scanning rules to automatically flag common vulnerabilities and misconfigurations.

Kali Linux

A Debian-based Linux distribution purpose-built for penetration testing and security research, used here as the virtual machine (via VirtualBox) that hosted DVWA, Burp Suite, and OWASP ZAP throughout the exercise.

4. Testing Environment Setup

The testing environment was set up inside a Kali Linux virtual machine running in Oracle VirtualBox, isolating all testing activity from the host system and any live network. DVWA was installed via the Kali package repository (`sudo apt install dvwa`) and started using the built-in `dvwa-start` service script, which launches the required Apache and MySQL/MariaDB services and prints the local URL to access the application.

- The database was initialised using DVWA's Setup/Reset DB page.
- **Login credentials used:** admin / password (DVWA's default credentials, intended only for this lab environment).
- **Security Level:** set to Low, to expose the vulnerabilities in their most basic, unmitigated form for demonstration purposes.

```
(kali@kali)-[~]
$ sudo dvwa-start

(Message from Kali developers)

Please wait for the dvwa service to start

[*] Web UI: http://127.0.0.1:42001
[i] You might need to refresh your browser once it opens

ERROR: Missing DVWA database
ERROR: Need to complete the first time setup to initialise database (after logging into DVWA)
- Default User      : admin
- Default Password : password
```

Fig 4.1 — DVWA service started via the Kali `dvwa-start` command, exposing the local Web UI.

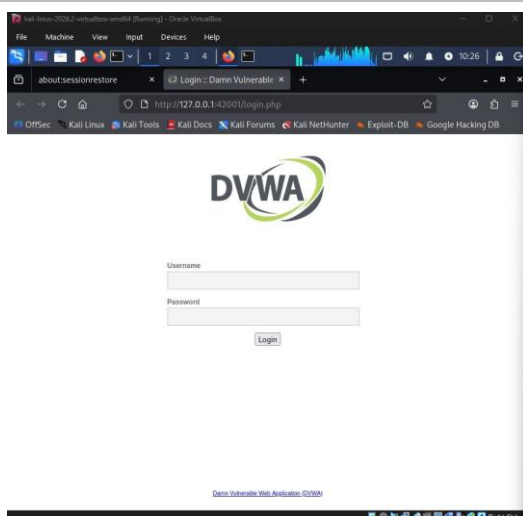


Fig 4.2 — DVWA login page.

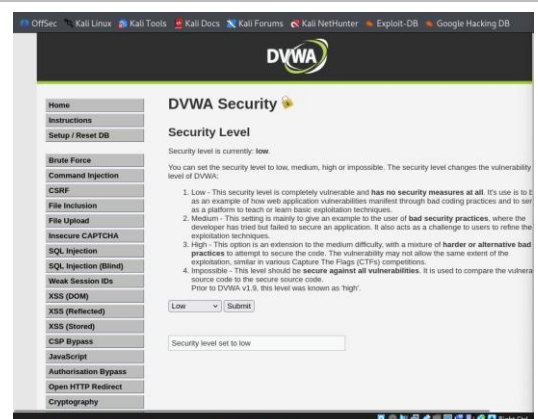


Fig 4.3 — DVWA Security Level set to Low.

5. Testing Process

This section documents each vulnerability that was tested, the payload used, the tool-level evidence captured, and the observed result. All testing was performed exclusively against the local DVWA instance (<http://127.0.0.1:42001>), which is explicitly designed and provided for this purpose.

5.1 SQL Injection Testing

The DVWA "SQL Injection" module accepts a User ID and displays the matching user's first name and surname. With Burp Suite's proxy intercepting traffic, the request generated by submitting a normal ID was first captured and examined, and then the same field was used to inject SQL Injection payloads.

Step 1 — Baseline request and interception

A normal value (User ID = 1) was submitted with Burp Suite's Intercept switched on, capturing the raw HTTP GET request before it reached the server.

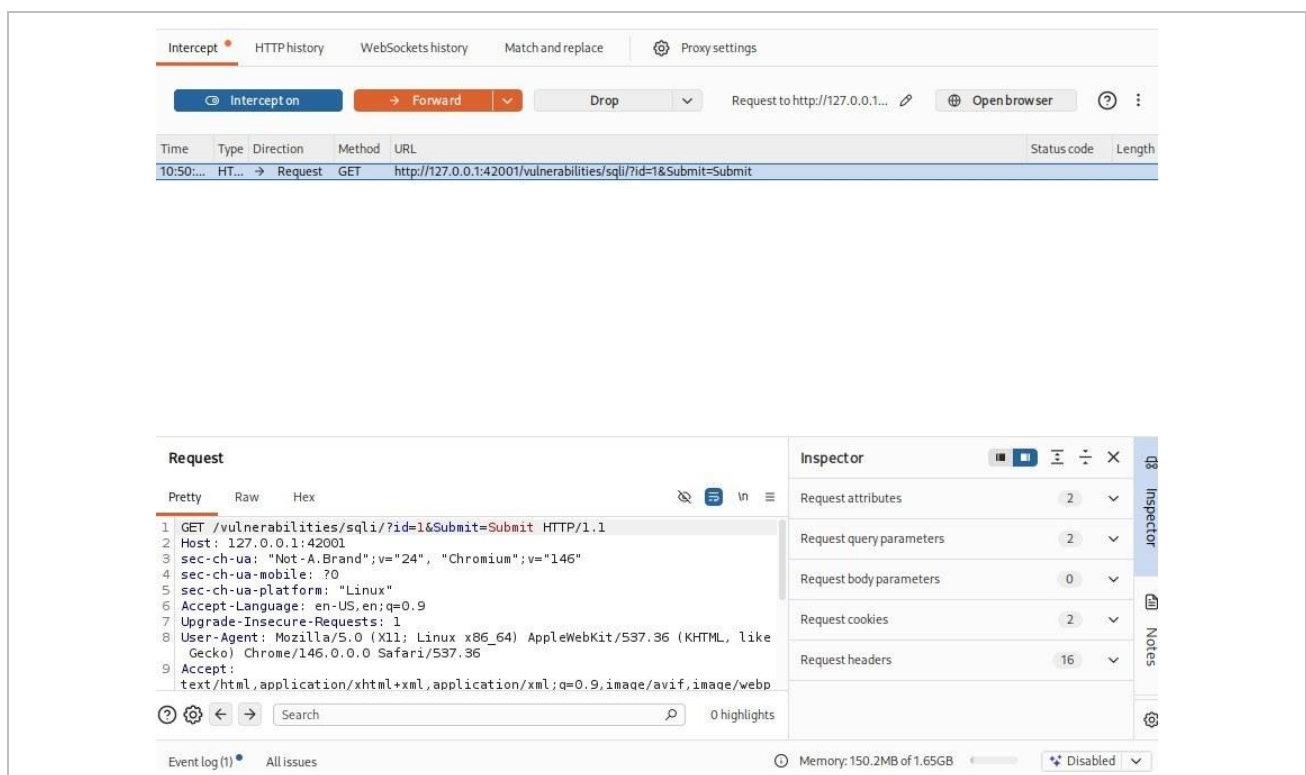


Fig 5.1 — Burp Suite Proxy intercepting the request: `GET /vulnerabilities/sqli/?id=1&Submit=Submit`



Fig 5.2 — Request tab in Burp's HTTP History.

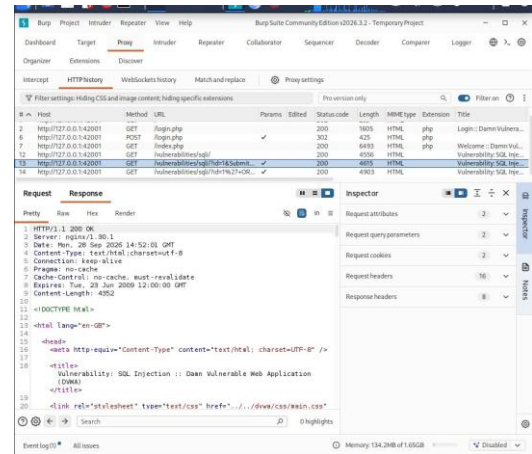


Fig 5.3 — Corresponding Response tab (HTTP 200 OK).

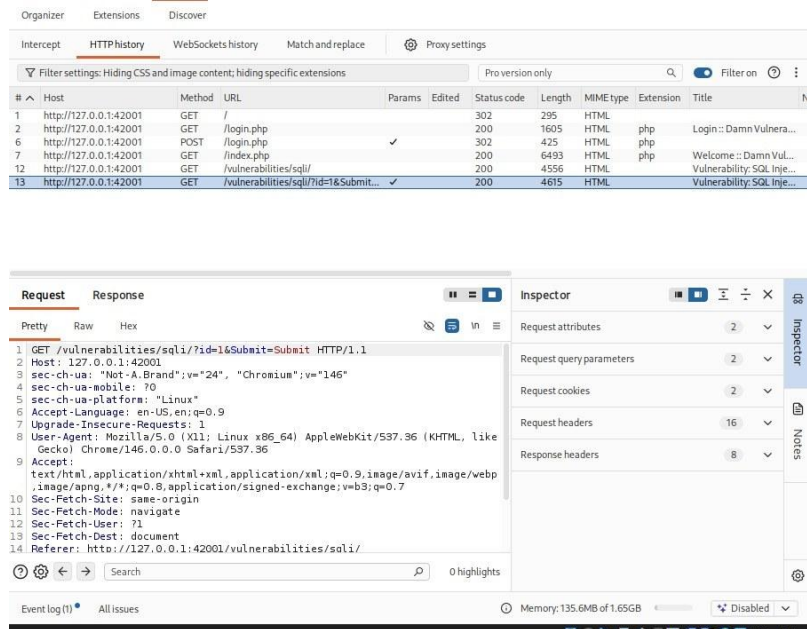


Fig 5.4 — Baseline result returned by the application: ID 1 → First name: admin, Surname: admin.

Step 2 — Exploiting the vulnerability

The following classic authentication-bypass / data-dump payload was then submitted in place of a normal ID:

1' OR '1'='1

Because the injected condition '1'='1' is always true, the WHERE clause in the underlying SQL query was neutralised, causing the query to match and return every row in the users table instead of the single intended record.

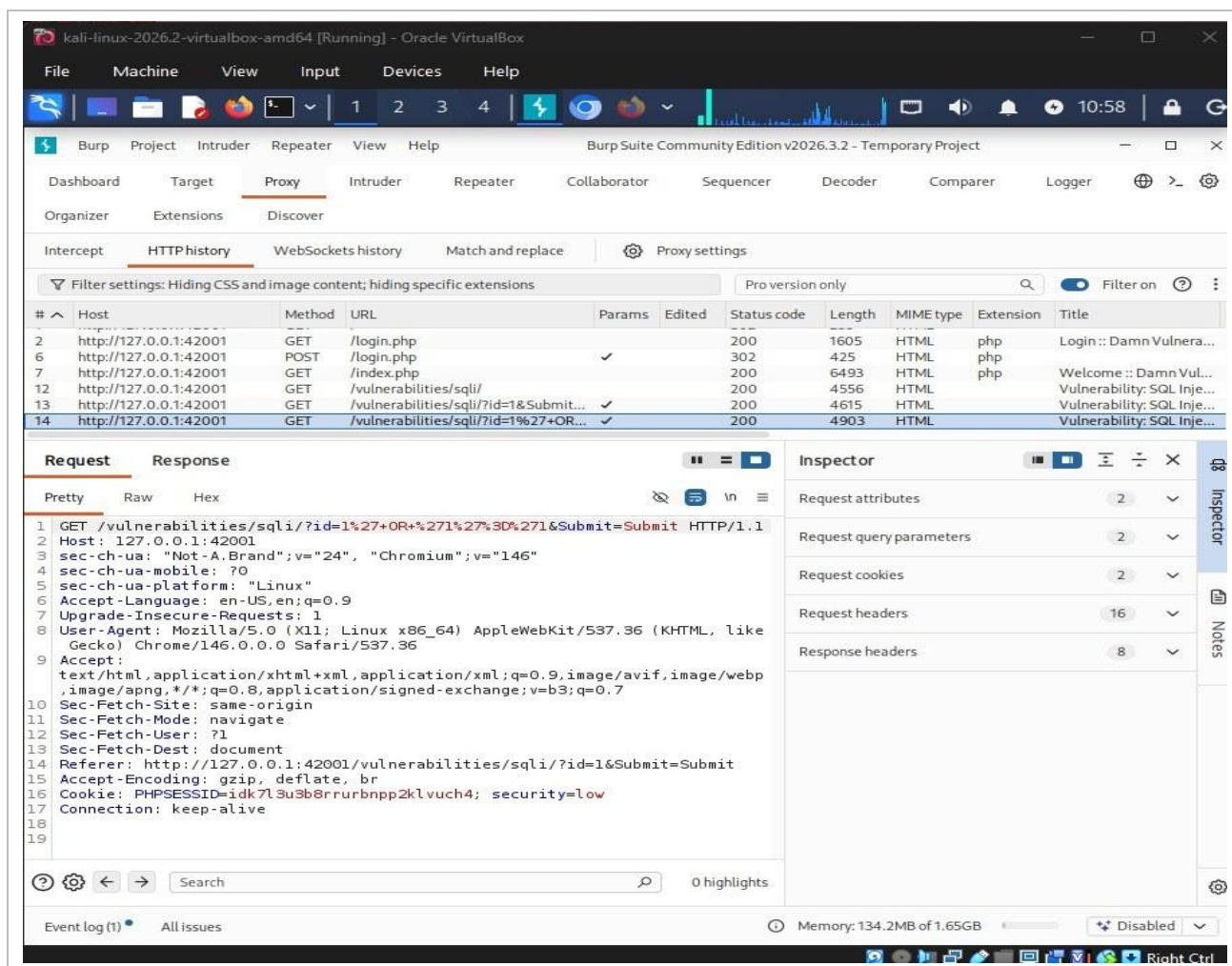


Fig 5.5 — Burp Suite capturing the injected request: id=1%27+OR+%271%27%3D%271

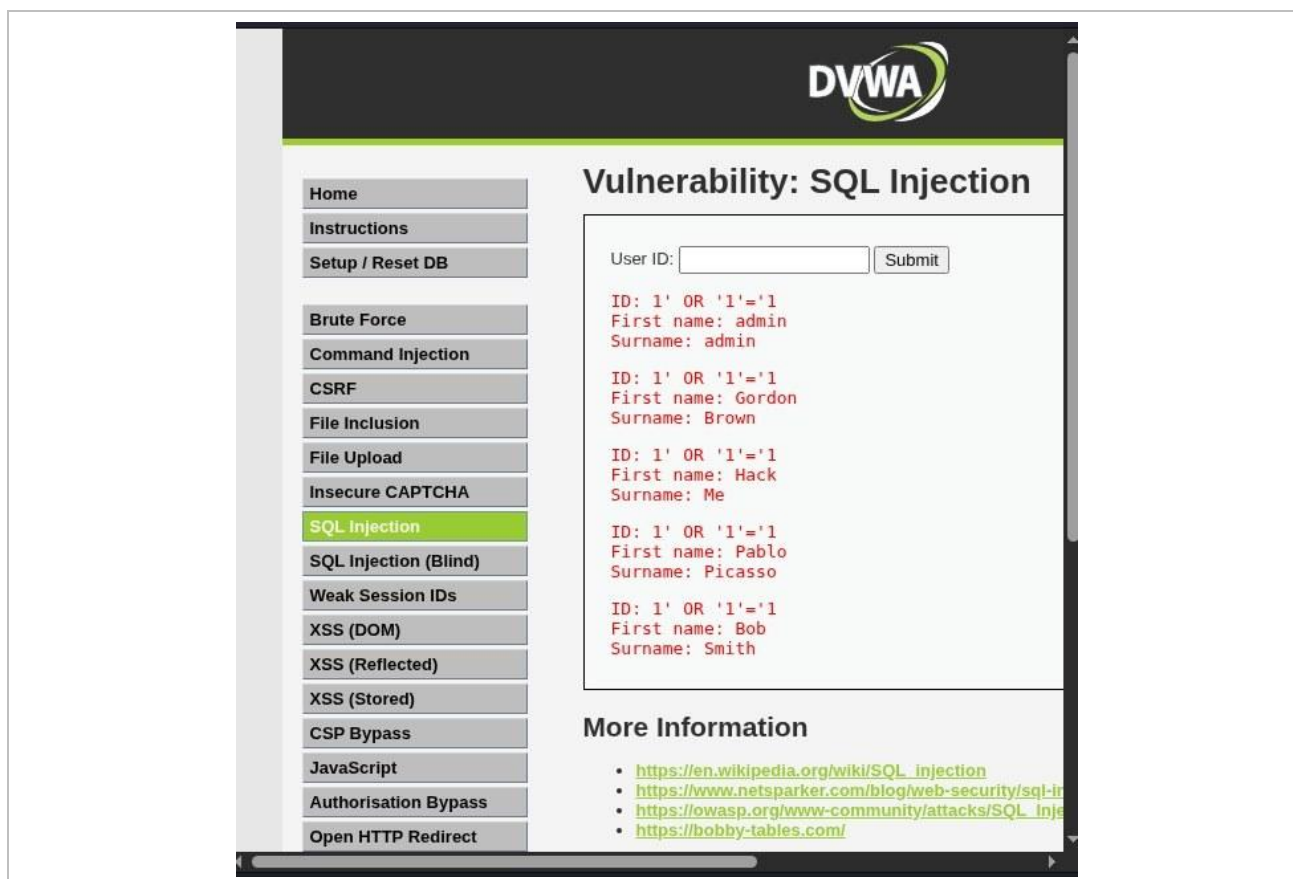


Fig 5.6 — Exploit result: all five user records (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith) disclosed in a single response — confirming the SQL Injection vulnerability.

Result: The SQL Injection vulnerability was successfully confirmed. An attacker with no valid credentials could enumerate the entire user table, and — as shown by the more advanced UNION-based technique available in the same module — could similarly retrieve usernames and password hashes directly from the database.

5.2 Cross-Site Scripting (XSS) — Reflected

The DVWA "XSS (Reflected)" module asks the user for their name and echoes it back on the page. Since the input is reflected directly into the HTML response without sanitisation or encoding, any script submitted in the field is executed by the browser exactly as if it were part of the page.

The following payload was submitted in the input field:

```
<script>alert('XSS')</script>
```

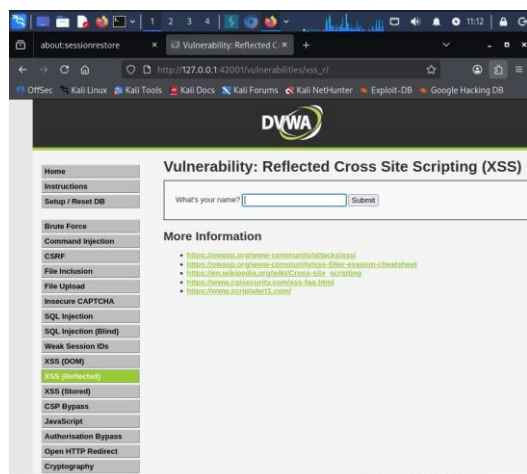


Fig 5.7 — XSS (Reflected) input page before injection.

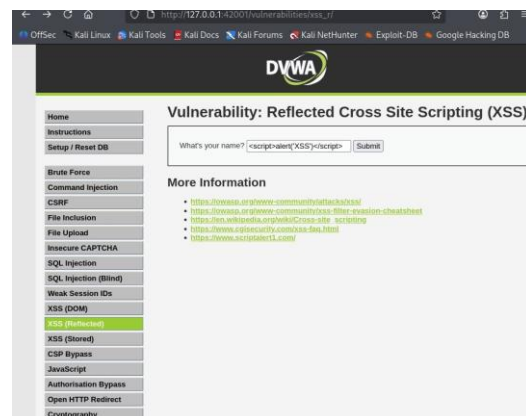


Fig 5.8 — Payload entered into the "What's your name?" field.

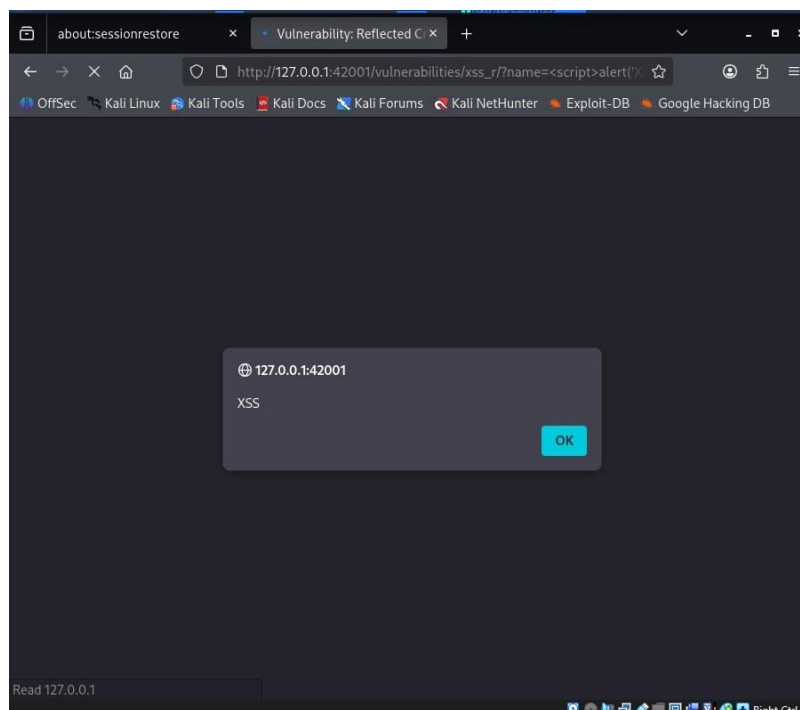


Fig 5.9 — JavaScript alert box "XSS" triggered immediately on submission, confirming the injected script executed in the victim's browser.

Result: Reflected XSS was confirmed. In a real attack scenario, this payload would instead be embedded in a crafted link sent to a victim; the script would run in the victim's session and could be used to steal cookies or redirect them to a malicious page.

5.3 Cross-Site Scripting (XSS) — Stored

The DVWA "XSS (Stored)" module implements a simple guestbook where a Name and Message are permanently saved to the database and displayed to every visitor of the page. Unlike reflected XSS, a stored payload does not require tricking a specific victim into clicking a link — it silently affects every user who later views the page.

The following payload was submitted in the Message field:

```
<script>alert('Stored XSS')</script>
```

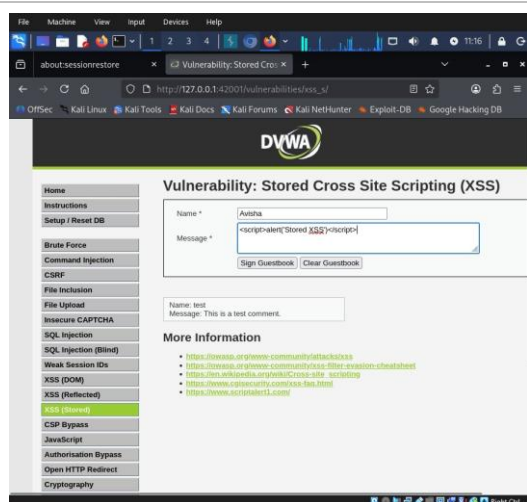


Fig 5.10 — Guestbook form with the Stored XSS payload entered in the Message field.

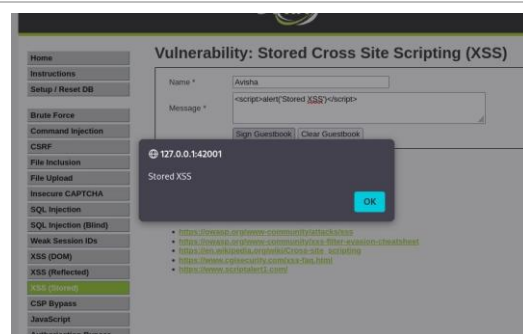


Fig 5.11 — Alert box "Stored XSS" fired immediately on submission.

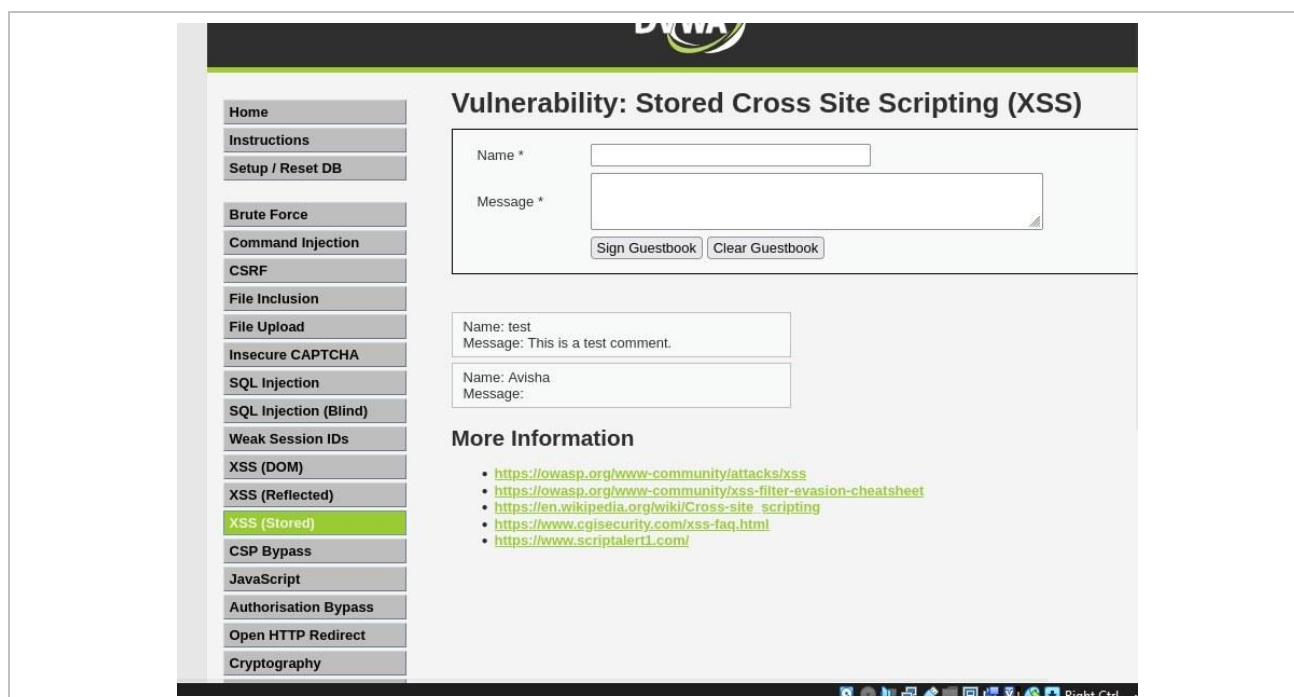


Fig 5.12 — Guestbook entry saved to the database; the alert re-fires on every page reload, confirming the payload is persistently stored rather than merely reflected.

Result: Stored XSS was confirmed. Because the payload persists in the database, it poses a higher risk than reflected XSS — every subsequent visitor to the guestbook page would have the malicious script executed in their browser without any additional action by the attacker.

5.4 Automated Scanning with OWASP ZAP

In addition to the manual tests above, an automated scan was run against the DVWA instance using OWASP ZAP's "Automated Scan" feature, which spiders the target application and then runs ZAP's built-in library of passive and active scan rules against every discovered page and parameter.

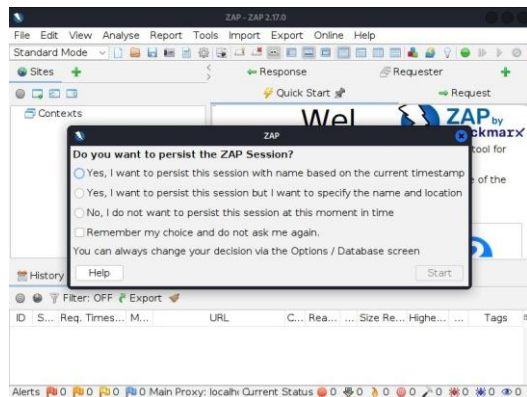


Fig 5.13 — New ZAP session started (no persistence, for a one-off lab scan).

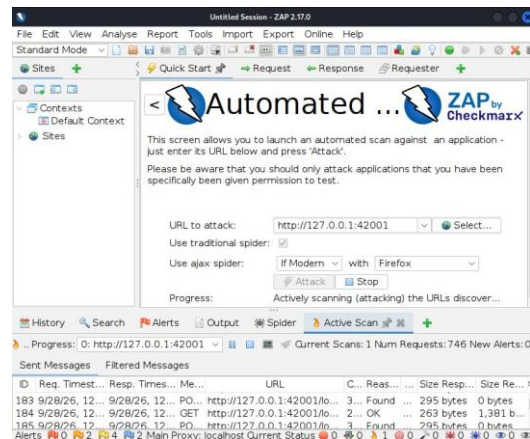


Fig 5.14 — Active Scan in progress against `http://127.0.0.1:42001`.

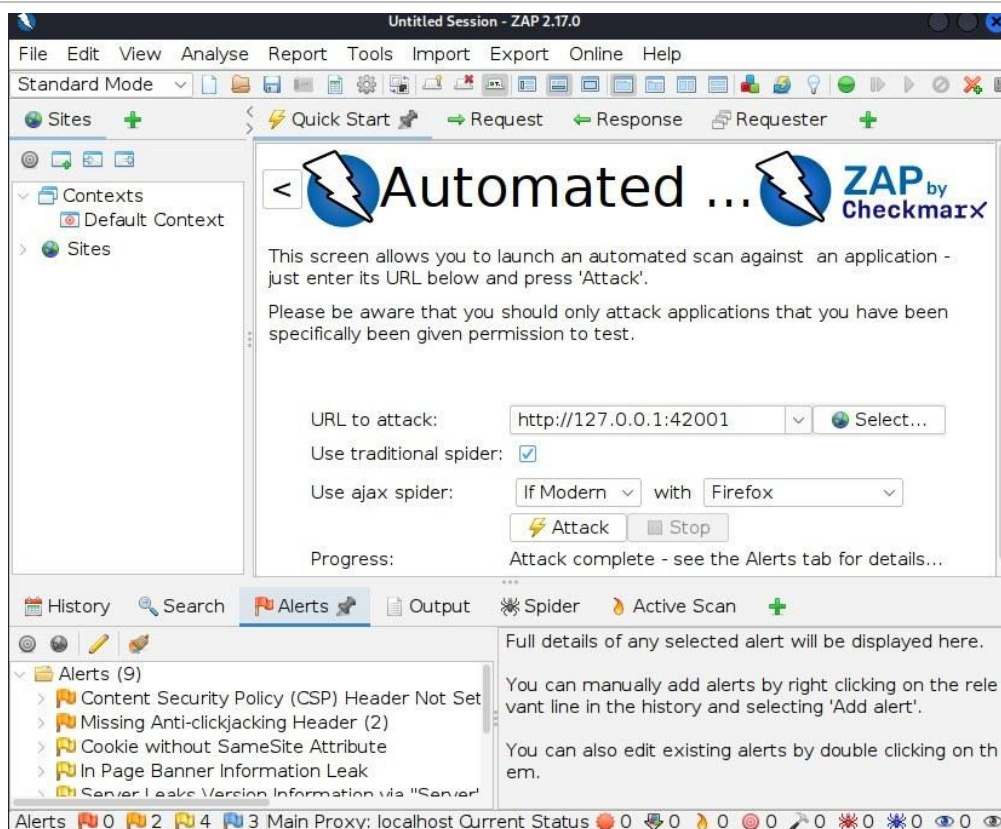


Fig 5.15 — Scan complete: OWASP ZAP raised 9 alerts, visible in the Alerts tab.

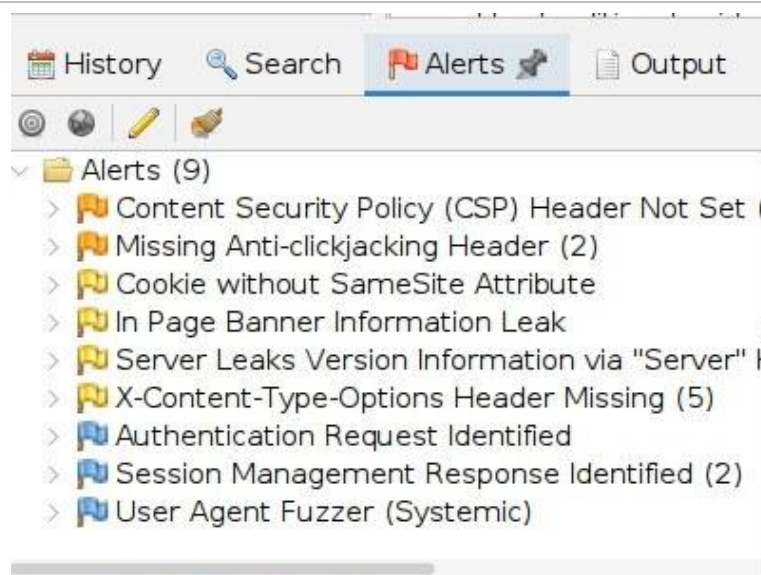


Fig 5.16 — Full list of the 9 alerts identified by the automated scan.

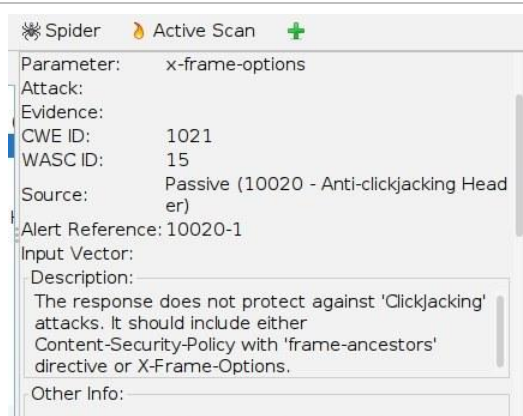


Fig 5.17 — Detail view: Missing Anti-clickjacking Header (CWE-1021).

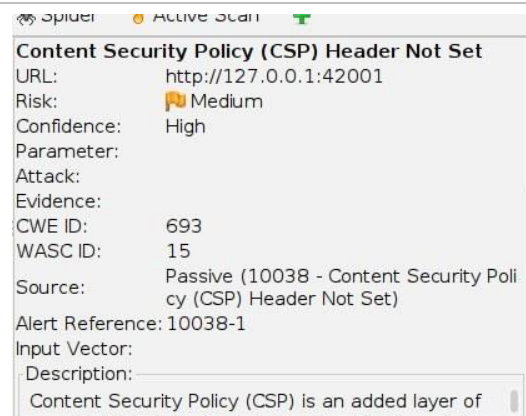


Fig 5.18 — Detail view: Content Security Policy (CSP) Header Not Set (CWE-693), Risk: Medium.



Fig 5.19 — Recommended Solution and Reference links for the CSP finding, as provided by ZAP.

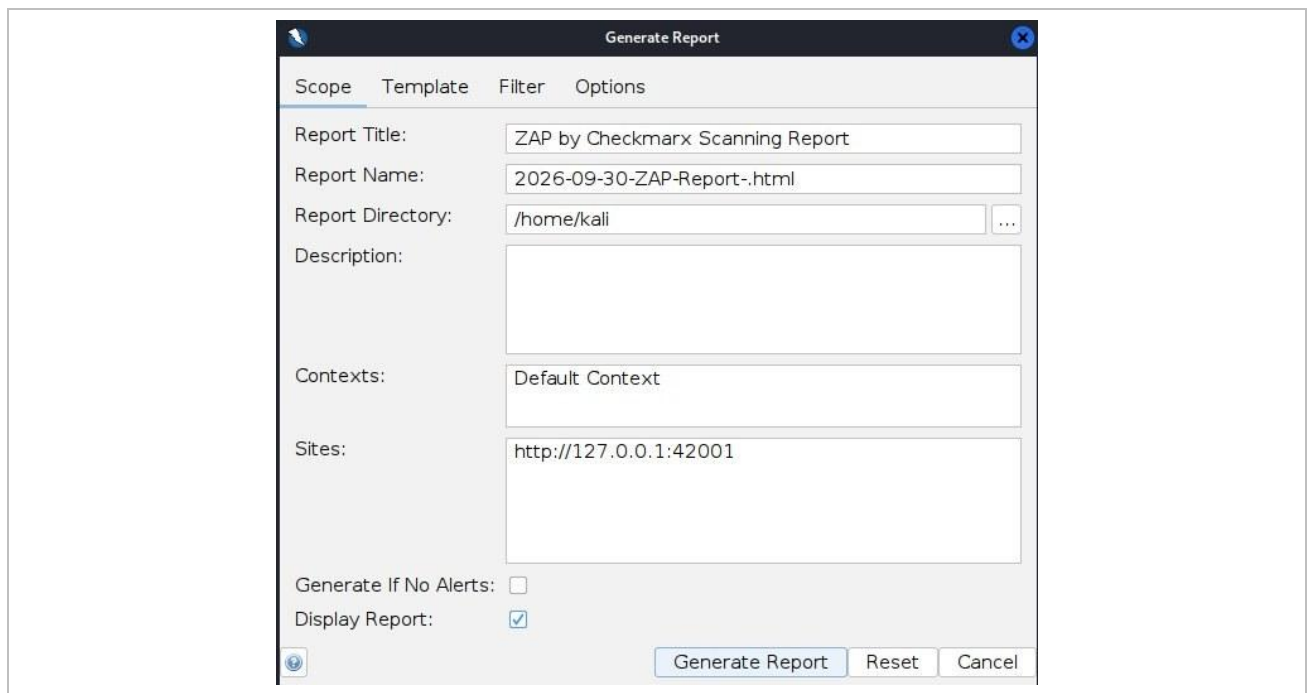


Fig 5.20 — Generating the final ZAP HTML/PDF scan report for submission.

6. Findings & Risk Levels

The table below consolidates the vulnerabilities confirmed through manual testing with Burp Suite and the alerts raised by the OWASP ZAP automated scan, with a risk level assigned to each.

Manually Confirmed Vulnerabilities (Burp Suite)

Vulnerability	Location	Risk	Impact Summary
SQL Injection	User ID field, /vulnerabilities/sqli/	Critical	Full user table (and, via UNION, credential hashes) disclosed with no authentication.
Cross-Site Scripting (Reflected)	"What's your name?" field, /vulnerabilities/xss_r/	High	Arbitrary JavaScript executes in the victim's browser via a crafted link.
Cross-Site Scripting (Stored)	Guestbook Message field, /vulnerabilities/xss_s/	High	Malicious script persists in the database and executes for every visitor.

OWASP ZAP Automated Scan Findings (9 Alerts)

Alert Type	Risk Level	Instances
Content Security Policy (CSP) Header Not Set	Medium	3
Missing Anti-clickjacking Header (X-Frame-Options)	Medium	2
Server Leaks Version Information via "Server" Header	Low	5
X-Content-Type-Options Header Missing	Low	5
Cookie without SameSite Attribute	Low	1
In Page Banner Information Leak	Low	1
Session Management Response Identified	Informational	2
Authentication Request Identified	Informational	1
User Agent Fuzzer	Informational	5

Overall, the automated scan reported 0 High-risk, 2 Medium-risk, 4 Low-risk, and 3 Informational alert categories (9 alerts in total) across the 8 endpoints crawled on the target site — consistent with a deliberately under-hardened lab application. The two manually confirmed vulnerabilities (SQL Injection and both forms of XSS) were not flagged as separately labelled alerts by the passive/default automated scan configuration, which highlights an important VAPT principle: automated scanning is efficient at catching configuration-level issues (missing headers, information leakage) but manual, logic-aware testing remains essential for catching injection-class vulnerabilities that depend on understanding the application's business logic.

7. Mitigation Methods

SQL Injection

- Use parameterised queries / prepared statements (or an ORM) instead of concatenating user input into SQL strings.
- Apply strict server-side input validation and enforce least-privilege database accounts.
- Disable verbose database error messages in production.

Cross-Site Scripting (Reflected & Stored)

- Perform context-aware output encoding of all user-supplied data before rendering it into HTML/JavaScript.
- Set a strong Content-Security-Policy (CSP) header to restrict script execution sources.
- Mark session cookies as HttpOnly and Secure so they cannot be read by injected scripts.

Security Misconfiguration (ZAP findings)

- Explicitly configure the Content-Security-Policy and X-Frame-Options (or frame-ancestors) headers to prevent clickjacking and restrict content sources.
- Add the X-Content-Type-Options: nosniff header to prevent MIME-type sniffing attacks.
- Set the SameSite attribute on all cookies to reduce exposure to cross-site request forgery.
- Suppress or minimise the Server header and any banner text that discloses exact software versions.
- Keep the web server, application framework, and all dependencies patched and up to date.

8. Conclusion

This VAPT exercise successfully demonstrated the end-to-end process of assessing a web application for security weaknesses: setting up an isolated test environment, performing manual exploitation of injection-class vulnerabilities (SQL Injection and both Reflected and Stored XSS) using Burp Suite, and running a complementary automated scan with OWASP ZAP to catch configuration-level issues such as missing security headers and information disclosure.

The exercise reinforced a key lesson in application security: automated tools are valuable for broad, repeatable coverage of known configuration and header-level issues, but they cannot substitute for manual testing when it comes to vulnerabilities that depend on understanding how an application processes and trusts user input. A mature VAPT methodology — and a mature development process — therefore combines both approaches, alongside secure coding practices such as parameterised queries, output encoding, and properly configured HTTP security headers, to meaningfully reduce a web application's attack surface.

— End of Report —